

Panduan dosen - Lab 03 Networking & Web Services

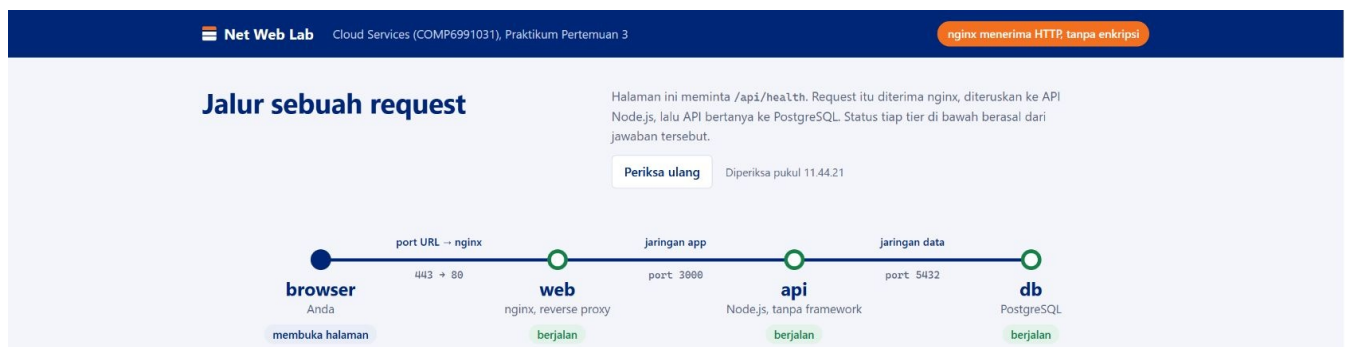
COMP6991031 | Pertemuan 3 | 120 menit Repo kelas: [SeedFlora/meet3CloudService](#) (**Public template**). Materi pendamping: [modul mahasiswa](#), [panduan Git](#), dan [template laporan](#).

Hasil belajar dan batas latihan

Mahasiswa menelusuri request browser -> nginx (web) -> Node API (api) -> PostgreSQL (db) dalam aplikasi empat container; memakai `dig`, `nslookup`, `ping`, `traceroute`, `curl`, `wget`, `nmap`, `nc`, dan `openssl`; mengidentifikasi port yang terpapar; lalu mengaktifkan HTTPS dan membatasi publikasi port tanpa merusak komunikasi antartier.

Pemindaian hanya untuk **service lab milik mahasiswa sendiri**.

Lab ini pengantar desain multi-tier. Docker Compose dipakai sebagai lingkungan konsisten, sedangkan detail container dibahas di Lab 04-06. Pekerjaan kode mahasiswa berada di `compose.yaml`, `web/nginx/https.conf`, dan lima fungsi TODO pada `scripts/healthcheck.sh`. Mereka tidak perlu membuat frontend/backend baru.



Langkah UI demo: buka port 8080 dari panel **Ports** Codespaces. Browser meminta halaman ke nginx; JavaScript meminta `/api/health`, nginx meneruskan ke API, lalu API mengecek PostgreSQL. Tunjuk empat status hijau. Label `443 -> 80` adalah forwarding HTTPS browser ke HTTP nginx starter, belum TLS nginx.

Persiapan dosen

1. Pastikan repo kelas menampilkan **Public template** dan tombol **Use this template**. Minta mahasiswa membuat repo pribadi dari template, lalu membuat Codespace dari repo masing-masing. Jalur cadangan: Docker Desktop/Engine lokal, Compose plugin, dan Git Bash/WSL di Windows.
2. Pastikan root repo mahasiswa memuat `.devcontainer/`, `.github/workflows/`, `compose.yaml`, `MODUL_MAHASISWA.md`, dan `screenshots/`. Tunggu `postCreateCommand` Codespaces selesai. Jika pull image terputus, ulangi `docker compose up -d --build --wait`.
3. Cek Docker dan port host 8080, 3000, 5432. Bila port bentrok, hentikan service lokal yang memakainya. Siapkan repo demo terpisah tanpa data pribadi.
4. Tampilkan panel **Ports** Codespaces. Gunakan visibilitas **Private** untuk port praktik. Port 8443 memakai sertifikat self-signed, sehingga browser mungkin memberi peringatan.

Terangkan status Actions awal: `stack` dirancang hijau; `student` merah selama `student.json` masih contoh; `challenge` **sengaja merah** sebelum tugas selesai.

Rencana kelas 120 menit

Waktu	Kegiatan	Bukti mahasiswa
0-10 menit	Uraikan browser -> web -> API -> DB; buat repo pribadi dan Codespace/clone lokal.	URL repo dan terminal di root.
10-25 menit	Preflight, hidupkan stack, buka web, jalankan smoke.	<code>docker compose ps</code> , port 8080, <code>smoke.sh</code> .
25-45 menit	DNS dan konektivitas dari toolbox; bahas <code>localhost</code> host vs container.	<code>dig</code> , <code>nslookup</code> , <code>ping</code> / <code>traceroute</code> .
45-65 menit	HTTP 200/201/404/400, header proxy, dan <code>wget</code> .	Output <code>curl -i</code> , perbandingan <code>viaProxy</code> .
65-80 menit	Pindai hanya stack sendiri; jalankan challenge awal.	<code>nmap/nc</code> , port host awal, challenge merah.
80-105 menit	Sertifikat, HTTPS, segmentasi, dan fungsi healthcheck.	TLS, port akhir, <code>healthcheck.sh</code> .
105-120 menit	Uji ulang, laporan, cek staged diff, commit/push, lihat Actions.	Repo/commit, screenshot pribadi, CI.

Jika sesi 90 menit, kerjakan checkpoint 1-4 di kelas dan lanjutkan checkpoint 5 beserta laporan sebagai tugas. Tetap kumpulkan bukti port **sebelum dan sesudah** perubahan.

Demo dan checkpoint

1. Stack awal dan alur request

Dari root repo demo:

```
docker compose version
docker compose config -q
docker compose up -d --build --wait
docker compose ps
bash tests/smoke.sh
```

Tunjukkan `web`, `api`, `db` sehat serta `toolbox` berjalan. Pada paket uji lokal, smoke awal memberi **12 PASS, 0 FAIL**. Buka port 8080 pada panel **Ports** Codespaces atau `http://localhost:8080` secara lokal. Kolom **PORTS** starter memperlihatkan `web`, `API`, dan `DB` mem-publish port host.

Docker Compose · labs/03/starter · kondisi awal

```
$ docker compose ps # kolom service, status, ports
SERVICE STATUS PORTS
api Up 2 minutes (healthy) 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
db Up 2 minutes (healthy) 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
toolbox Up 2 minutes
web Up 2 minutes (healthy) 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

*Perintah: `docker compose ps`. Compose meminta status runtime dan port publish dari Docker Engine. Tunjuk **healthy** pada `web/API/DB`, **running** pada `toolbox`, serta mapping host 8080/3000/5432 pada starter. Minta mahasiswa membandingkan output mereka, bukan menyalin gambar.*

Gambar berikut adalah hasil **uji live di GitHub Codespaces**: empat service berjalan dan smoke **12 PASS, 0 FAIL**. Halaman web di pembuka menampilkan empat tier aktif. Pakai ini untuk menunjukkan bahwa jalur Codespaces sungguh dapat menjalankan stack. Tautan browser Codespaces dapat memakai HTTPS karena port forwarding GitHub; itu **belum membuktikan** nginx starter telah mengaktifkan TLS. Bukti tantangan adalah `https://web` dari toolbox dengan `--cacert` dan sertifikat yang diperiksa OpenSSL.

```

@SeedFlora → /workspaces/meet3CloudService (main) $ docker compose ps
netlab-web-1      public.ecr.aws/docker/library/nginx:1.30-alpine   "/docker-entrypoint..." web      3 minutes ago   Up 3 minutes (healthy)   0.0.0.0:8080->80/tcp
80/tcp
@SeedFlora → /workspaces/meet3CloudService (main) $ docker compose ps --format '{{.Service}} {{.State}} {{.Health}}' && bash tests/smoke.sh | tail -n 4
api running healthy
db running healthy
toolbox running
web running healthy
4. Dari host (port yang dipublish service web)
PASS http://127.0.0.1:8080/api/health -> 200

smoke: 12 PASS, 0 FAIL
@SeedFlora → /workspaces/meet3CloudService (main) $

```

Perintah live: `docker compose ps --format '{{.Service}} {{.State}} {{.Health}}' && bash tests/smoke.sh | tail -n 4`. Compose merangkum empat container; && menjalankan smoke hanya setelah Compose berhasil. Smoke meminta DNS, halaman, API, POST/GET DB, dan akses host; `tail -n 4` menyisakan empat baris terakhir. Tunjuk smoke: 12 PASS, 0 FAIL; minta mahasiswa menjalankan smoke tanpa tail untuk bukti lengkap.

2. DNS dan konektivitas internal

```

docker compose exec toolbox bash
dig web A
nslookup api
getent hosts db
ping -c 3 web
traceroute -m 5 web

```

Minta mahasiswa menyebutkan **nama service, IP internal, jaringan, dan posisi terminal**. web dan api dapat punya lebih dari satu IP karena terhubung ke beberapa jaringan. localhost di toolbox menunjuk toolbox, bukan laptop. Jika ICMP dibatasi, gunakan TCP/HTTP sebagai pembuktian tambahan dan catat batasannya.

Docker Compose · labs/03/starter · kondisi awal

```

$ docker compose exec -T toolbox dig +short web A
172.28.20.4

$ docker compose exec -T toolbox getent hosts api db
172.28.20.3      api
172.28.30.2      db

$ docker compose exec -T toolbox ping -c 2 web
PING web (172.28.20.4) 56(84) bytes of data.
64 bytes from netlab-web-1.netlab_app (172.28.20.4): icmp_seq=1 ttl=64 time=0.069 ms
64 bytes from netlab-web-1.netlab_app (172.28.20.4): icmp_seq=2 ttl=64 time=0.057 ms

--- web ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1007ms
rtt min/avg/max/mdev = 0.057/0.063/0.069/0.006 ms

```

Perintah: `dig web A`, `nslookup api`, dan `ping -c 3 web` dari toolbox. DNS Compose mengubah nama service menjadi IP internal; ping menguji ICMP. Tunjuk alamat 172.28.x.x dan jumlah paket diterima/loss.

```

@SeedFlora →/workspaces/meet3CloudService (main) $
@SeedFlora →/workspaces/meet3CloudService (main) $ docker compose exec -T toolbox bash -lc 'echo DNS; dig +short web; nslookup api | tail -n 4; echo PING; ping -c 1 web | tail -n 2; echo TRACE; traceroute -m 3 web | head -n 2'
172.28.20.4
Non-authoritative answer:
Name:     api
Address:  172.28.20.3

PING
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.055/0.055/0.055/0.000 ms
TRACE
traceroute to web (172.28.20.4), 3 hops max, 60 byte packets
 1 netlab-web-1.netlab_app (172.28.20.4)  0.035 ms  0.021 ms  0.018 ms
@SeedFlora →/workspaces/meet3CloudService (main) $

```

Perintah live di gambar: `docker compose exec -T toolbox bash -lc '...' -T` mematikan terminal interaktif, `bash -lc` menjalankan beberapa perintah di toolbox, `;` memisahkan perintah, dan `| tail -n` merangkum output. Di dalamnya `dig +short web/nslookup api` meminta IP internal, `ping -c 1 web` menguji ICMP, dan `traceroute -m 3 web` melihat hop. Tunjuk IP web/API, 0% packet loss, dan hop menuju web; mahasiswa dapat mencoba perintah satu per satu di shell toolbox.

3. HTTP dan reverse proxy

Masih di toolbox:

```

curl -i http://web/api/health
curl -i http://web/api/whoami
curl -i http://web/api/notes/999999999
curl -i -X POST http://web/api/notes -H 'Content-Type: application/json' -d '{"text": "catatan kelas"}'
wget -qO- http://web/api/health

```

Tunjukkan status dan header sebelum body JSON. Health memberi 200 dan db: up; catatan yang tidak ada 404; POST valid 201 dengan Location. Untuk 400, kirim JSON rusak atau lihat smoke test. Bandingkan `curl -i` (header dan body) dengan `wget -qO-` (body). Dari host, bandingkan request ke 8080 melalui nginx dengan 3000 langsung ke API dan field `viaProxy`.

Pada uji browser **Codespaces live**, form frontend berhasil menyimpan catatan baru dan menampilkan 201 Created; daftar serta hitungan catatan PostgreSQL bertambah. Gunakan observasi ini untuk menunjukkan POST browser -> nginx -> API -> DB. Screenshot area bawah web tidak disertakan karena memuat alamat IP klien.

```

Docker Compose - labs/03/starter - kondisi awal

$ docker compose exec -T toolbox curl -sS -i http://web/api/health
HTTP/1.1 200 OK
Server: nginx/1.30.5
Date: Thu, 01 Oct 2026 00:47:23 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 214
Connection: keep-alive
Cache-Control: no-store

[
  "status": "ok",
  "service": "api",
  "db": "up",
  "hostname": "58fee130d498",
  "time": "2026-10-01T00:47:23.497Z",
  "db_version": "PostgreSQL 17.11",
  "notes": 3,
  "node": "v22.23.3",
  "uptime_s": 125
]

$ docker compose exec -T toolbox curl -sS -i http://web/api/notes/999999999
HTTP/1.1 404 Not Found
Server: nginx/1.30.5
Date: Thu, 01 Oct 2026 00:47:23 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 73
Connection: keep-alive
Cache-Control: no-store

[
  "error": "not_found",
  "message": "Catatan #999999999 tidak ada."
]

```

Perintah: `curl -i http://web/api/health` dan `curl -i http://web/api/notes/999999999`. Curl menampilkan status/header/body dari nginx dan API. Tunjuk 200 dengan db: up serta 404 untuk ID tak ada; hubungkan POST valid ke 201 dan Location.

4. Port dan challenge awal

```
nmap -sT -Pn -p 8080,3000,5432 host.docker.internal
nc -vz -w 2 host.docker.internal 5432
exit
bash tests/challenge.sh
```

Challenge starter sengaja gagal. Contoh paket uji awal menunjukkan **1 PASS, 11 FAIL**; jumlah bisa berbeda menurut kondisi mesin. Bahas beda port yang dipublish ke host dengan koneksi internal jaringan `app/data`.

Docker Compose · labs/03/starter · kondisi awal

```
$ docker compose exec -T toolbox nmap -sT -Pn -p 8080,3000,5432 host.docker.internal
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-01 00:47 UTC
Nmap scan report for host.docker.internal (192.168.65.254)
Host is up (0.0027s latency).
Other addresses for host.docker.internal (not scanned): fdc4:f303:9324::254

PORT      STATE SERVICE
3000/tcp  open  ppp
5432/tcp  open  postgresql
8080/tcp  open  http-proxy

Nmap done: 1 IP address (1 host up) scanned in 0.04 seconds
```

Perintah: `nmap -sT -Pn -p 8080,3000,5432 host.docker.internal`. Toolbox mencoba koneksi TCP ke host Docker. Tunjuk tiga status open dan cocokkan dengan mapping pada docker compose ps; target pemindaian hanya lab sendiri.

```
@SeedFlora →/workspaces/meet3CloudService (main) $ docker compose exec -T toolbox bash -lc 'curl -s -o /dev/null -w "HTTP %{http_code}" http://web/api/health; wget -qO- http://web/api/health; echo; nmap -sT -Pn -p 3000,5432,8080 host.docker.internal | tail -n 6; nc -vz -w 2 api 3000; openssl version'
HTTP 200
{
  "status": "ok",
  "service": "api",
  "db": "up",
  "hostname": "92134bceede0",
  "time": "2026-10-01T04:41:09.404Z",
  "db_version": "PostgreSQL 17.11",
  "notes": 5,
  "node": "v22.23.3",
  "uptime_s": 285
}

PORT      STATE SERVICE
3000/tcp  open  ppp
5432/tcp  open  postgresql
8080/tcp  open  http-proxy

Nmap done: 1 IP address (1 host up) scanned in 0.03 seconds
Connection to api (172.28.20.3) 3000 port [tcp/*] succeeded!
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
@SeedFlora →/workspaces/meet3CloudService (main) $
```

Perintah live dibungkus `docker compose exec -T toolbox bash -lc '...';` menjalankan alat berurutan dan `| tail -n` membatasi output nmap yang tampak. `curl` mengambil status health, `wget -qO-` mengambil JSON, `nmap` mencoba port host, `nc -vz api 3000` menguji satu socket, dan `openssl version` memeriksa alat TLS. Tunjuk HTTP 200, db: up, port 3000/5432/8080 open, dan koneksi API succeeded sebagai dasar perubahan desain.

```

tests/challenge.sh pada repo awal (sengaja gagal)

~/net-web-lab$ bash tests/challenge.sh
A. Desain di compose.yaml (docker compose config)
FAIL service yang mem-publish port ke host: api db web (seharusnya hanya web)
-> api mem-publish 3000:3000: hapus bagian ports: dari service api
-> db mem-publish 5432:5432: hapus bagian ports: dari service db
FAIL web belum mem-publish port 443 (HTTPS)
-> compose.yaml, service web: ports: tambahkan "8443:443"
PASS web dan db tidak berada di jaringan yang sama (segmentasi)
info bonus (opsional): jadikan jaringan data internal: true

B. Sertifikat web/certs/lab.crt
FAIL sertifikat belum ada (web/certs/lab.crt dan lab.key)
-> buat dengan: bash scripts/make-cert.sh

C. HTTPS dari toolbox (https://web)
FAIL curl --cacert lab.crt https://web/api/health -> 000
-> aktifkan web/nginx/https.conf (salin dari https.conf.example), lalu: docker compose up -d --wait
FAIL openssl s_client ke web:443: tidak ada koneksi TLS

D. Exposure: port yang bisa dihubungi dari host
FAIL port 3000 terbuka di host (127.0.0.1:3000)
-> hapus publish port 3000 di compose.yaml (atau hentikan aplikasi lain yang memakai port itu)
FAIL port 3000 terbuka bila dilihat dari toolbox lewat host.docker.internal
FAIL port 5432 terbuka di host (127.0.0.1:5432)
-> hapus publish port 5432 di compose.yaml (atau hentikan aplikasi lain yang memakai port itu)
FAIL port 5432 terbuka bila dilihat dari toolbox lewat host.docker.internal

E. Segmentasi: container web tidak boleh menghubungi db
FAIL web BISA membuka koneksi ke port 5432: lewat IP 172.28.30.2; lewat host 172.28.20.1
-> db mem-publish 5432: container di jaringan mana pun bisa masuk lewat host Docker (172.28.20.1)
-> IP db terjangkau dari jaringan lain (terjadi di Docker Desktop selama 5432 dipublish)

F. scripts/healthcheck.sh (dijalankan di toolbox)
| Net Web Lab healthcheck (2026-09-30 07:52:07 UTC)
|
| == 1. DNS: nama service ter-resolve oleh DNS internal Docker
| [... dipotong ...]
|
| Ringkasan: 0 lolos, 0 gagal, 5 TODO
| Masih ada 5 fungsi bertanda [TODO]. Lengkapi fungsinya, lalu jalankan lagi.
FAIL healthcheck.sh keluar dengan kode 1
-> lengkapi fungsi check_* yang masih [TODO] / perbaiki yang [GAGAL]

challenge: 1 PASS, 11 FAIL
(kode keluar: 1)

```

Perintah: `bash tests/challenge.sh`. Skrip membandingkan desain Compose, sertifikat, HTTPS, exposure, segmentasi, dan healthcheck dengan sasaran lab. Tunjuk FAIL beserta hint sebagai diagnosis awal; bukan error platform. Jalankan ulang setelah tugas untuk membuktikan semua lulus.

5. TLS dan perubahan desain

Minta mahasiswa membuat sertifikat dengan `bash scripts/make-cert.sh`, menyalin `web/nginx/https.conf.example` ke `web/nginx/https.conf`, dan mengubah `compose.yaml` agar hanya `web` mem-publish port host 8080/8443. `web` <-> `api` tetap di jaringan `app`; `api` <-> `db` tetap di `data`. `web` tidak perlu berbagi jaringan langsung dengan `db`. Mahasiswa melengkapi `check_dns`, `check_tcp`, `check_http`, `check_tls`, dan `check_exposure` di `scripts/healthcheck.sh`; komentar `TODO` adalah petunjuk. Jangan bagikan file solusi penuh.

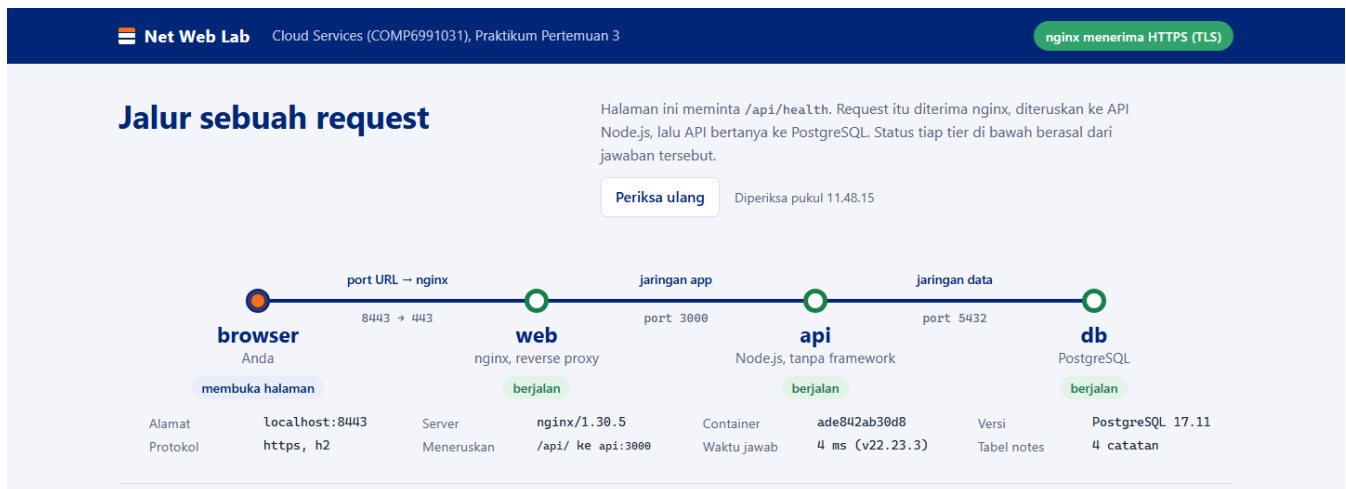
```

docker compose up -d --force-recreate --wait
docker compose exec toolbox curl -i --cacert /lab/web/certs/lab.crt https://web/api/health
docker compose exec toolbox openssl x509 -in /lab/web/certs/lab.crt -noout -subject -issuer -dates
\ext subjectAltName
docker compose exec toolbox bash scripts/healthcheck.sh
bash tests/smoke.sh
bash tests/challenge.sh
docker compose ps

```

Sasaran: HTTPS menjawab 200 dengan `--cacert`; sertifikat memuat SAN `web` dan masih berlaku; smoke hijau; healthcheck keluar 0 dengan minimal 10 pemeriksaan lolos; challenge hijau (contoh paket uji **15 PASS, 0 FAIL**). Hanya `web` mem-publish port host. API/DB tetap berjalan pada jaringan internal.

Hasil acuan dari uji end-to-end **lokal** pada paket ini: starter empat service berjalan, smoke **12/0**, challenge awal **1/11**; setelah penyelesaian sementara, smoke **12/0**, healthcheck **16/0**, challenge **15/0**, verifikasi OpenSSL **0 (ok)**, TLS **1.3**, dan hanya host 8080/8443 terbuka. Starter juga diuji **live di Codespaces SeedFlora** dengan empat service sehat dan smoke **12/0**. File solusi sementara tidak disertakan pada repo template. Akses dan kuota Codespaces setiap akun mahasiswa tetap perlu dicek.



Perintah: docker compose up -d --force-recreate --wait, lalu docker compose exec toolbox curl --cacert /lab/web/certs/lab.crt https://web/api/health. Curl memverifikasi sertifikat lab serta koneksi nginx:443 dan harus memberi 200. Saat membuka https://localhost:8443 lokal, tunjuk label 8443 -> 443, badge HTTPS, dan empat tier hijau; browser dapat memperingatkan sertifikat self-signed.

Catatan

Tersimpan di PostgreSQL (tabel notes). Form ini mengirim POST `/api/notes`, jadi datanya melewati ketiga tier.

Tulis catatan

Misalnya: halo dari browser **Simpan catatan**

0/280 Catatan #4 tersimpan (201 Created).

#	Detail	Tanggal
#4	Catatan ditambah lewat browser HTTPS untuk verifikasi Lab 03.	1 Okt 2026, 11.48
#3	Tambahkan catatan lewat form di halaman ini atau dengan curl -X POST ke <code>/api/notes</code> .	1 Okt 2026, 11.47
#2	Alur request: browser -> nginx (web) -> Node.js (api) -> PostgreSQL (db).	1 Okt 2026, 11.47
#1	Halo! Tiga catatan ini dibuat oleh db/init.sql saat database pertama kali dibuat.	1 Okt 2026, 11.47

*Langkah UI: isi form **Tulis catatan** dan klik **Simpan catatan**. Browser mengirim POST `/api/notes` melalui nginx ke API; API menulis ke PostgreSQL. Tunjuk 201 Created dan catatan baru sebagai bukti jalur tulis tiga tier tetap berfungsi setelah TLS/segmentasi.*

Docker Compose - target akhir Lab 03

```
$ docker compose ps --format 'table [$.Service]]$t[$.Status]]$t[$.Ports]]'
SERVICE STATUS PORTS
api Up About a minute (healthy) 3000/tcp
db Up About a minute (healthy) 5432/tcp
toolbox Up About a minute
web Up About a minute (healthy) 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp, 0.0.0.0:8443->443/tcp, [::]:8443->443/tcp
```

Perintah pada gambar: docker compose ps --format 'table {\$.Service}}\t{\$.Status}}\t{\$.Ports}}' setelah Compose direcreate. Tunjuk web mem-publish 8080 -> 80 dan 8443 -> 443, sedangkan API/DB hanya port internal dan tetap healthy. Bandingkan dengan nmap awal serta bash tests/smoke.sh dan bash tests/challenge.sh akhir. Gambar target berasal dari salinan uji, bukan bukti mahasiswa.

Panduan dosen: menuntun challenge.sh sampai tuntas

Bagian ini adalah urutan mengajar dan mendiagnosis, bukan file jawaban untuk disalin mahasiswa. Jalankan `bash tests/challenge.sh` dari **root repo** pada **terminal host/Codespaces**, karena skrip memanggil Docker Compose. Jalankan `scripts/healthcheck.sh` di **toolbox**. Challenge membaca keadaan sistem; ia tidak memperbaiki file otomatis.

0. Ambil bukti sebelum perubahan

```
docker compose config -q
docker compose up -d --build --wait
docker compose ps
bash tests/smoke.sh
bash tests/challenge.sh
echo $?
```

`config -q` memeriksa YAML dan hasil interpolasi Compose tanpa output bila valid. `up --wait` menunggu healthcheck container. `smoke.sh` memastikan fungsi dasar tetap bekerja sebelum desain diubah (paket uji: 12 PASS). `challenge.sh` biasanya keluar bukan 0 pada starter; angka PASS/FAIL dapat berubah jika mahasiswa sudah membuat sertifikat atau ada port lain di komputer. Minta screenshot **output awal milik mahasiswa**, bukan angka tertentu. `echo $?` menampilkan exit code perintah terakhir di Bash: 0 berarti seluruh pemeriksaan challenge lulus.

Skrip memuat helper dari `tests/lib.sh`, lalu membagi pemeriksaan ke **A-F**. A membaca `docker compose config --format json`; B-F memerlukan stack hidup. Jika host belum memiliki `jq`, helper memakai `jq` di toolbox. Jangan menyimpulkan masalah desain dari pesan `compose.yaml` tidak valid: betulkan sintaks YAML terlebih dahulu.

```
tests/challenge.sh pada repo awal (sengaja gagal)

~/net-web-lab$ bash tests/challenge.sh
A. Desain di compose.yaml (docker compose config)
FAIL service yang mem-publish port ke host: api db web (seharusnya hanya web)
-> api mem-publish 3000:3000: hapus bagian ports: dari service api
-> db mem-publish 5432:5432: hapus bagian ports: dari service db
FAIL web belum mem-publish port 443 (HTTPS)
-> compose.yaml, service web: ports: tambahkan "8443:443"
PASS web dan db tidak berada di jaringan yang sama (segmentasi)
info bonus (opsional): jadikan jaringan data internal: true

B. Sertifikat web/certs/lab.crt
FAIL sertifikat belum ada (web/certs/lab.crt dan lab.key)
-> buat dengan: bash scripts/make-cert.sh

C. HTTPS dari toolbox (https://web)
FAIL curl --cacert lab.crt https://web/api/health -> 000
-> aktifkan web/nginx/https.conf (salin dari https.conf.example), lalu: docker compose up -d --wait
FAIL openssl s_client ke web:443: tidak ada koneksi TLS

D. Exposure: port yang bisa dihubungi dari host
FAIL port 3000 terbuka di host (127.0.0.1:3000)
-> hapus publish port 3000 di compose.yaml (atau hentikan aplikasi lain yang memakai port itu)
FAIL port 3000 terbuka bila dilihat dari toolbox lewat host.docker.internal
FAIL port 5432 terbuka di host (127.0.0.1:5432)
-> hapus publish port 5432 di compose.yaml (atau hentikan aplikasi lain yang memakai port itu)
FAIL port 5432 terbuka bila dilihat dari toolbox lewat host.docker.internal

E. Segmentasi: container web tidak boleh menghubungi db
FAIL web BISA membuka koneksi ke port 5432: lewat IP 172.28.30.2; lewat host 172.28.20.1
-> db mem-publish 5432: container di jaringan mana pun bisa masuk lewat host Docker (172.28.20.1)
-> IP db terjangkau dari jaringan lain (terjadi di Docker Desktop selama 5432 dipublish)

F. scripts/healthcheck.sh (dijalankan di toolbox)
| Net Web Lab healthcheck (2026-09-30 07:52:07 UTC)
|
| == 1. DNS: nama service ter-resolve oleh DNS internal Docker
| [... dipotong ...]
|
| Ringkasan: 0 lolos, 0 gagal, 5 TODO
| Masih ada 5 fungsi bertanda [TODO]. Lengkapi fungsinya, lalu jalankan lagi.
FAIL healthcheck.sh keluar dengan kode 1
-> lengkapi fungsi check_* yang masih [TODO] / perbaiki yang [GAGAL]

challenge: 1 PASS, 11 FAIL
(kode keluar: 1)
```

Langkah: Jalankan `bash tests/challenge.sh` sebelum mengubah starter. Fungsi: Menetapkan diagnosis awal A-F. Cara kerja: Tes membaca Compose lalu mencoba sertifikat, HTTPS, exposure, segmentasi, dan healthcheck. Baca hasil: Baris FAIL beserta hint menjadi pekerjaan mahasiswa; contoh uji bersih memberi 1 PASS/11 FAIL, tetapi jumlah awal dapat berubah.

1. Baca hasil A: desain Compose (3 PASS)

1. **Publisher host:** hanya web boleh punya bagian `ports: . api:3000` dan `db:5432` tetap bisa dihubungi lewat jaringan internal walaupun tidak dipublish ke host. `expose:` hanya dokumentasi port internal dan tidak menggantikan `ports: .`

- Port TLS:** web harus memetakan sebuah port host ke **port container 443**, contoh host 8443 ke container 443. HTTP 8080 ke 80 boleh tetap ada agar perbandingan awal/akhir terlihat.
- Segmentasi:** web dan db tidak boleh berbagi jaringan. Hubungan yang diperlukan adalah web- api di app, serta api-db di data; toolbox boleh bergabung ke ketiga jaringan sebagai alat uji. `data.internal: true` adalah bonus, bukan syarat 15 PASS.

Minta mahasiswa membaca baris `service` yang mem-publish port ke host, web belum mem-publish port 443, dan web dan db sama-sama terhubung satu per satu. Setelah edit `compose.yaml`, ulangi `docker compose config -q`. Nilai desain harus dibuktikan lagi pada runtime; menghapus `ports`: di file tanpa membuat ulang container dapat meninggalkan mapping lama.

Docker Compose - target akhir Lab 03

```
$ docker compose ps --format 'table [$.Service]]$t[$.Status]]$t[$.Ports]]'
SERVICE STATUS PORTS
api Up About a minute (healthy) 3000/tcp
db Up About a minute (healthy) 5432/tcp
toolbox Up About a minute
web Up About a minute (healthy) 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp, 0.0.0.0:8443->443/tcp, [::]:8443->443/tcp
```

Langkah: Jalankan `docker compose up -d --force-recreate --wait` lalu `docker compose ps`. Fungsi: Memastikan perubahan Compose benar-benar diterapkan. Cara kerja: Compose membuat ulang container dan membaca status/port dari Docker Engine. Baca hasil: Hanya web mem-publish host 8080/8443; API dan DB tetap healthy pada port internal.

```
A Compose: publikasi port dan segmentasi jaringan

$ docker compose ps --format 'table [$.Service]]$t[$.Status]]$t[$.Ports]]'
SERVICE STATUS PORTS
api Up 5 minutes (healthy) 3000/tcp
db Up 5 minutes (healthy) 5432/tcp
toolbox Up 5 minutes
web Up 5 minutes (healthy) 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp,
0.0.0.0:8443->443/tcp, [::]:8443->443/tcp
[exit 0]

$ docker network inspect netlab_edge --format '{{range .Containers}}{{.Name}} {{end}}'
netlab-toolbox-1 netlab-web-1
[exit 0]

$ docker network inspect netlab_app --format '{{range .Containers}}{{.Name}} {{end}}'
netlab-api-1 netlab-toolbox-1 netlab-web-1
[exit 0]

$ docker network inspect netlab_data --format '{{range .Containers}}{{.Name}} {{end}}'
netlab-api-1 netlab-db-1 netlab-toolbox-1
[exit 0]
```

Langkah: Jalankan `docker compose ps --format 'table [$.Service]]$t[$.Status]]$t[$.Ports]]'`, lalu `docker network inspect netlab_edge`, `docker network inspect netlab_app`, dan `docker network inspect netlab_data`. Fungsi: Mencocokkan port host serta container pada tiap jaringan. Cara kerja: Docker membaca metadata Compose yang sedang berjalan. Baca hasil: edge memuat web/toolbox; app memuat web/api/toolbox; data memuat api/db/toolbox, tanpa web. Gambar adalah keluaran uji lokal yang ditata ulang.

2. Baca hasil B: sertifikat (3 PASS)

```
bash scripts/make-cert.sh
docker compose exec toolbox openssl x509 -in /lab/web/certs/lab.crt -noout -subject -issuer -dates
\ -ext subjectAltName
```

Perintah pertama membuat sertifikat dan kunci lab. Perintah kedua membaca sertifikat tanpa membocorkan isi private key. Tunjuk tiga SAN wajib: `DNS:localhost`, `DNS:web`, dan `IP Address:127.0.0.1`. Challenge juga meminta

sertifikat masih berlaku **minimal tujuh hari**. Jika file hilang, SAN kurang, atau masa berlaku pendek, buat ulang dengan `make-cert.sh. web/certs/lab.key` tetap lokal dan tidak masuk Git.

●●● B Sertifikat HTTPS: SAN dan masa berlaku

```
$ docker compose exec -T toolbox openssl x509 -in /lab/web/certs/lab.crt -noout -subject
      -ext subjectAltName -dates
subject=CN = localhost
X509v3 Subject Alternative Name:
      DNS:localhost, DNS:web, IP Address:127.0.0.1
notBefore=Oct  1 04:32:25 2026 GMT
notAfter=Oct 31 04:32:25 2026 GMT
[exit 0]
```

Langkah: Jalankan `openssl x509 -in /lab/web/certs/lab.crt -noout -subject -ext subjectAltName -dates` dari toolbox. Fungsi: Membaca identitas dan masa berlaku sertifikat. Cara kerja: OpenSSL mengurai CRT publik tanpa menampilkan private key. Baca hasil: SAN memuat localhost, web, dan 127.0.0.1; notAfter masih di masa depan. Gambar adalah keluaran uji lokal yang ditata ulang agar terbaca.

●●● B Masa berlaku sertifikat > 7 hari

```
$ docker compose exec -T toolbox bash -lc 'openssl x509 -in /lab/web/certs/lab.crt
      -noout -checkend 604800; echo exit=$?'
Certificate will not expire
exit=0
[exit 0]
```

Langkah: Jalankan `openssl x509 -in /lab/web/certs/lab.crt -noout -checkend 604800` dari toolbox. Fungsi: Memastikan sisa masa berlaku minimal 604800 detik. Cara kerja: OpenSSL memberi exit code 0 bila sertifikat belum akan habis dalam tujuh hari. Baca hasil: Certificate will not expire dan exit 0. Gambar adalah keluaran uji lokal yang ditata ulang.

3. Baca hasil C: HTTPS nginx dari toolbox (2 PASS)

```
cp web/nginx/https.conf.example web/nginx/https.conf
docker compose up -d --force-recreate --wait
docker compose exec toolbox curl -i --cacert /lab/web/certs/lab.crt https://web/api/health
docker compose exec -T toolbox \
  openssl s_client -connect web:443 -servername web \
  -CAfile /lab/web/certs/lab.crt -verify_hostname web \
  </dev/null
```

Nginx memuat berkas berakhiran `.conf`; `.conf.example` sendiri belum mengaktifkan HTTPS. Buat sertifikat sebelum me-recreate web, karena konfigurasi TLS membutuhkannya. `curl --cacert` memvalidasi sertifikat self-signed lab dan mengirim GET melalui nginx ke API dan database; baca JSON `status: ok, db: up`, dan HTTP 200. OpenSSL harus menampilkan `Verify return code: 0 (ok)` untuk nama web. Jangan memakai `curl -k` sebagai bukti C: opsi itu melewati verifikasi sertifikat. HTTPS pada URL forwarding Codespaces juga belum membuktikan nginx port 443 aktif.

Jika status `000/connection refused`, cek `docker compose ps`, `docker compose logs web`, nama `https.conf`, dan sertifikat. Jika HTTPS menjawab `502/503/504`, periksa `docker compose logs api` dan kesehatan DB; sesudah perubahan jaringan gunakan `--force-recreate --wait`.

```

C HTTPS: reverse proxy /api/health

$ docker compose exec -T toolbox curl -sSi --cacert /lab/web/certs/lab.crt
https://web/api/health
HTTP/2 200
server: nginx/1.30.5
date: Thu, 01 Oct 2026 05:59:19 GMT
content-type: application/json; charset=utf-8
content-length: 214
cache-control: no-store

{
  "status": "ok",
  "service": "api",
  "db": "up",
  "hostname": "426a5490ed16",
  "time": "2026-10-01T05:59:19.653Z",
  "db_version": "PostgreSQL 17.11",
  "notes": 3,
  "node": "v22.23.3",
  "uptime_s": 153
}
[exit 0]

```

Langkah: Dari toolbox jalankan `curl -sSi --cacert /lab/web/certs/lab.crt https://web/api/health`. Fungsi: Memeriksa HTTPS dan API melalui nginx. Cara kerja: Curl memverifikasi sertifikat lab, lalu nginx meneruskan GET ke API/DB. Baca hasil: HTTP/2 200, status: ok, dan db: up. Gambar adalah keluaran uji lokal yang ditata ulang agar terbaca.

```

C TLS: nama host dan CA terverifikasi

$ docker compose exec -T toolbox bash -lc 'echo | openssl s_client -connect web:443
-sservername web -CAfile /lab/web/certs/lab.crt -verify_hostname web 2>&1 | tail -n
12'
Verified peername: web
---
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Server public key is 2048 bit
Secure Renegotiation IS NOT supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)
---
DONE
[exit 0]

```

Langkah: Jalankan `openssl s_client` dengan `-servername web`, `-CAfile /lab/web/certs/lab.crt`, dan `-verify_hostname web` dari toolbox. Fungsi: Menguji handshake TLS serta identitas server. Cara kerja: OpenSSL memeriksa rantai trust dan SAN untuk nama web. Baca hasil: TLSv1.3 dan Verify return code: 0 (ok); ini lebih kuat daripada `curl -k`. Gambar adalah keluaran uji lokal yang ditata ulang.

4. Baca hasil D: exposure host (5 PASS)

```

docker compose ps
curl -i -k https://127.0.0.1:8443/api/health
docker compose exec toolbox nmap -sT -Pn -p 8080,8443,3000,5432 host.docker.internal

```

Port host HTTPS yang sebenarnya mengikuti mapping Anda (contoh **8443**). `curl -k` di langkah ini hanya memeriksa endpoint host dengan sertifikat self-signed; validasi trust tetap memakai `--cacert` pada C. Target akhir: HTTPS host memberi 200, sedangkan 3000 dan 5432 **tertutup dari dua sudut**: 127.0.0.1 pada host dan `host.docker.internal` dari toolbox. Challenge menghitung satu PASS untuk host HTTPS dan empat PASS untuk

dua port dari dua sudut itu. Jika satu port masih open, cocokkan docker compose ps dengan ports: dan cek aplikasi lain yang memakai port yang sama.

Port host tertutup, API internal hidup

```
$ docker compose exec -T toolbox nmap -sT -Pn -p 8080,8443,3000,5432
host.docker.internal
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-01 05:59 UTC
Nmap scan report for host.docker.internal (192.168.65.254)
Host is up (0.0011s latency).
Other addresses for host.docker.internal (not scanned): fdc4:f303:9324::254

PORT      STATE SERVICE
3000/tcp  closed ppp
5432/tcp  closed postgresql
8080/tcp  open  http-proxy
8443/tcp  open  https-alt

Nmap done: 1 IP address (1 host up) scanned in 0.02 seconds
[exit 0]

$ docker compose exec -T toolbox nc -vz -w 2 api 3000
Connection to api (172.28.20.3) 3000 port [tcp/*] succeeded!
[exit 0]
```

Langkah: Dari toolbox pindai hanya host lab dengan `nmap -sT -Pn -p 8080,8443,3000,5432 host.docker.internal`, lalu coba `nc -vz api 3000`. Fungsi: Membandingkan port host dengan port internal. Cara kerja: Nmap mencoba socket TCP ke host Docker; netcat mencoba API pada jaringan app. Baca hasil: 8080/8443 open, 3000/5432 closed di host, tetapi API internal succeeded. Gambar adalah keluaran uji lokal yang ditata ulang.

5. Baca hasil E: segmentasi yang benar (1 PASS)

Challenge mencoba membuka db:5432 dari container web melalui nama db, setiap IP container DB, dan gateway host Docker. Ketiganya harus gagal. Pertanyaan penting untuk diskusi: walaupun DNS db tidak ter-resolve dari web, apakah DB yang masih dipublish pada host bisa dicapai melalui gateway? Pada desain awal, jalur host dapat melubangi segmentasi. Karena itu perubahan jaringan dan penghapusan publish DB harus diperiksa bersama. Toolbox tetap boleh menghubungi DB, karena toolbox sengaja berada di jaringan data.

Web terisolasi dari DB; toolbox masih terhubung

```
$ docker compose exec -T web sh -lc 'nc -vz -w 2 db 5432; echo exit=$?'
nc: bad address 'db'
exit=1
[exit 0]

$ docker compose exec -T web sh -lc 'nc -vz -w 2 172.28.30.2 5432; echo exit=$?'
nc: 172.28.30.2 (172.28.30.2:5432): Operation timed out
exit=1
[exit 0]

$ docker compose exec -T web sh -lc 'nc -vz -w 2 172.28.20.1 5432; echo exit=$?'
exit=1
[exit 0]

$ docker compose exec -T toolbox nc -vz -w 2 db 5432
Connection to db (172.28.30.2) 5432 port [tcp/*] succeeded!
[exit 0]

Alamat 172.28.x.x berasal dari subnet Docker Compose lab.
```

Langkah: Uji `nc -vz -w 2` dari web ke nama, IP, dan gateway DB; bandingkan `nc -vz -w 2 db 5432` dari toolbox. Fungsi: Membuktikan isolasi web tanpa memutus DB dari alat uji/API. Cara kerja: Netcat mencoba handshake TCP dari dua container berbeda. Baca hasil: `exit=1` di dalam shell web berarti tiga koneksi gagal; `succeeded` dari toolbox berarti DB tetap hidup. `[exit 0]` di luar shell web hanya kode proses pembungkus echo. Gambar adalah keluaran uji lokal yang ditata ulang.

6. Baca hasil F: lima fungsi healthcheck (1 PASS)

```
docker compose exec toolbox bash scripts/healthcheck.sh
echo $?
```

Mahasiswa melengkapi lima fungsi bertanda TODO di `scripts/healthcheck.sh`: DNS untuk tiga nama service, TCP untuk `web:80`, `web:443`, `api:3000`, `db:5432`, HTTP health melalui proxy, TLS beserta masa berlaku/hostname, dan exposure host. Fungsi ok menambah penghitung lolos, bad menambah gagal, todo menandai fungsi belum dikerjakan. Target challenge: exit code **0**, baris Ringkasan: **N** lolos, **0** gagal, **0** TODO, dan **N** minimal **10** (penyelesaian paket uji: 16). Exit 0 dengan 9 lolos tetap gagal F. Jika ada [TODO] atau [GAGAL], gunakan nama section untuk kembali ke fungsi yang tepat.

```
F Healthcheck: 16 lolos

$ docker compose exec -T toolbox bash scripts/healthcheck.sh
Net Web Lab healthcheck (2026-10-01 05:59:27 UTC)

== 1. DNS: nama service ter-resolve oleh DNS internal Docker
[ OK ] web -> 172.28.20.4
[ OK ] api -> 172.28.20.3
[ OK ] db -> 172.28.30.2

== 2. TCP: port layanan terbuka di jaringan internal
[ OK ] web:80 terbuka
[ OK ] web:443 terbuka
[ OK ] api:3000 terbuka
[ OK ] db:5432 terbuka

== 3. HTTP: API sehat bila diakses lewat reverse proxy
[ OK ] HTTP health -> 200
[ OK ] API dan database sehat

== 4. TLS: HTTPS aktif dan sertifikatnya valid untuk nama web
[ OK ] sertifikat ada
[ OK ] sertifikat tervalidasi untuk web
[ OK ] sertifikat berlaku minimal tujuh hari
[ OK ] HTTPS health -> 200

== 5. Exposure: dari host Docker hanya port web yang terbuka
[ OK ] HTTPS 8443 terbuka di host
[ OK ] 3000 tertutup di host
[ OK ] 5432 tertutup di host

Ringkasan: 16 lolos, 0 gagal, 0 TODO
Semua pemeriksaan lolos.
[exit 0]
```

Langkah: Jalankan `docker compose exec -T toolbox bash scripts/healthcheck.sh`. Fungsi: Memeriksa DNS, TCP, HTTP, TLS, dan exposure sekaligus. Cara kerja: Lima fungsi memberi [OK] hanya bila pemeriksaan jaringan sesungguhnya lulus. Baca hasil: Ringkasan: 16 lolos, 0 gagal, 0 TODO dan exit 0 pada salinan uji dosen. Gambar adalah keluaran uji lokal yang ditata ulang.

7. Uji regresi dan simpulkan 15 PASS

```
docker compose up -d --force-recreate --wait
docker compose exec toolbox bash scripts/healthcheck.sh
bash tests/smoke.sh
bash tests/challenge.sh
docker compose ps
```

Hitung hasil akhir menurut kelompok: **A 3 + B 3 + C 2 + D 5 + E 1 + F 1 = 15 PASS**. Smoke harus tetap hijau (paket uji: 12 PASS, 0 FAIL), karena menutup port host API/DB tidak boleh memutus jalur internal browser -> nginx -> API -> PostgreSQL. Pada `docker compose ps`, hanya `web` mempunyai mapping host 8080/8443; `api` dan `db` tetap healthy. Mahasiswa menyimpan screenshot **awal dan akhir**, menjelaskan perubahan yang menyebabkan tiap FAIL berubah menjadi PASS, lalu commit/push. Jika challenge masih merah, baca kelompok gagal; jangan mengulang semua perubahan secara acak.

●●● Tantangan akhir: bagian A-C

```
$ bash tests/challenge.sh
A. Desain di compose.yaml (docker compose config)
PASS hanya service web yang mem-publish port ke host
PASS web mem-publish HTTPS: host 8443 -> container 443
PASS web dan db tidak berada di jaringan yang sama (segmentasi)
info bonus (opsional): jadikan jaringan data internal: true

B. Sertifikat web/certs/lab.crt
PASS web/certs/lab.crt dan lab.key ada
PASS SAN sertifikat: DNS:localhost, DNS:web, IP Address:127.0.0.1
PASS sertifikat masih berlaku minimal 7 hari lagi

C. HTTPS dari toolbox (https://web)
PASS curl --cacert lab.crt https://web/api/health -> 200
PASS openssl s_client -verify_hostname web: Verify return code: 0 (ok)
info New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
[exit 0]
```

Langkah: Jalankan `bash tests/challenge.sh` dari root repo setelah perbaikan. Fungsi: Memeriksa desain, sertifikat, dan HTTPS tanpa mengganti skrip tes. Cara kerja: A membaca Compose; B memeriksa CRT/SAN/umur; C meminta health melalui TLS dan OpenSSL. Baca hasil: A 3 PASS, B 3 PASS, C 2 PASS; nama web tervalidasi. Gambar adalah cuplikan keluaran uji lokal yang ditata ulang.

●●● Tantangan akhir: cuplikan D-F

```
$ bash tests/challenge.sh # cuplikan keluaran D-F
D. Exposure: port yang bisa dihubungi dari host
PASS https://127.0.0.1:8443/api/health -> 200 (dari host)
PASS port 3000 tertutup di host
PASS port 3000 tertutup dari toolbox lewat host.docker.internal
PASS port 5432 tertutup di host
PASS port 5432 tertutup dari toolbox lewat host.docker.internal
E. Segmentasi: container web tidak boleh menghubungi db
PASS web tidak bisa membuka koneksi ke db:5432 (dicoba: db, 172.28.30.2, host 172.28.20.1)
F. scripts/healthcheck.sh (dijalankan di toolbox)
| Ringkasan: 16 lolos, 0 gagal, 0 TODO
PASS healthcheck.sh keluar dengan kode 0 (16 pemeriksaan lolos)
challenge: 15 PASS, 0 FAIL
[exit 0]

Cuplikan verbatim dari output; transkrip lengkap tersimpan terpisah.
```

Langkah: Lanjut baca keluaran `bash tests/challenge.sh` yang sama. Fungsi: Membuktikan exposure, isolasi web dari DB, dan healthcheck. Cara kerja: D mencoba port dari host/toolbox, E mencoba jalur web ke DB, F menjalankan healthcheck. Baca hasil: D 5 PASS, E 1 PASS, F 1 PASS; ringkasan challenge: 15 PASS, 0 FAIL dan healthcheck 16/0/0. Gambar adalah cuplikan keluaran uji lokal yang ditata ulang.

Langkah 6 Git: perubahan solusi dan private key

```
$ git status --short
M compose.yaml
M scripts/healthcheck.sh
M web/html/index.html
M web/html/js/app.js
?? web/nginx/https.conf
[exit 0]

$ git check-ignore web/certs/lab.key
web/certs/lab.key
[exit 0]
```

Path lab.key pada check-ignore berarti Git mengecualikannya dari commit.

Langkah: Jalankan `git status --short` dan `git check-ignore web/certs/lab.key` sebelum commit. Fungsi: Meninjau file yang akan ikut Git dan memastikan private key dikecualikan. Cara kerja: Git membaca working tree dan aturan ignore. Baca hasil: Path lab.key keluar dari check-ignore; daftar file berubah pada salinan uji dapat berbeda dari pekerjaan mahasiswa dan bukan daftar wajib.

Selesai praktik: kontainer dan network dihentikan

```
$ docker compose down
Container netlab-toolbox-1 Stopping
Container netlab-web-1 Stopping
Container netlab-toolbox-1 Stopped
Container netlab-toolbox-1 Removing
Container netlab-web-1 Stopped
Container netlab-web-1 Removing
Container netlab-toolbox-1 Removed
Container netlab-web-1 Removed
Container netlab-api-1 Stopping
Container netlab-api-1 Stopped
Container netlab-api-1 Removing
Container netlab-api-1 Removed
Container netlab-db-1 Stopping
Container netlab-db-1 Stopped
Container netlab-db-1 Removing
Container netlab-db-1 Removed
Network netlab_app Removing
Network netlab_edge Removing
Network netlab_data Removing
Network netlab_data Removed
Network netlab_edge Removed
Network netlab_app Removed
[exit 0]

$ docker compose ps
NAME          IMAGE          COMMAND          SERVICE    CREATED   STATUS    PORTS
[exit 0]
```

Volume database dipertahankan karena perintah down tanpa -v.

Langkah: Jalankan `docker compose down`, lalu `docker compose ps`. Fungsi: Menghentikan container demo setelah bukti tersimpan. Cara kerja: Compose menurunkan container/jaringan tetapi mempertahankan volume database karena tidak memakai -v. Baca hasil: `docker compose ps` kosong. Gambar adalah keluaran uji lokal yang ditata ulang.

Penilaian

Komponen	Bobot	Yang diperiksa
Observasi awal	20%	Screenshot hasil sendiri: Compose, smoke, DNS/HTTP, challenge dan port awal; penjelasan status tepat.
Jaringan dan TLS	30%	Hanya web terpublikasi; API/DB sehat; HTTPS tervalidasi; tidak ada jalur web langsung ke DB.
Healthcheck dan uji ulang	25%	Lima fungsi bekerja, minimal 10 lolos, smoke dan challenge akhir lulus.
Penjelasan konsep	15%	Enam jawaban modul menjelaskan DNS, TCP, proxy, HTTP, segmentasi, dan trust sertifikat.
Pengumpulan	10%	<code>student.json</code> , laporan, screenshot pribadi, commit/push, dan Actions dapat diperiksa; tanpa kunci privat.

Nilai didasarkan pada hasil, bukan pilihan Codespaces atau Docker lokal. Jika kuota Codespaces terbatas, terima jalur lokal dengan bukti setara. Screenshot modul yang disalin bukan bukti praktik.

Masalah umum

Gejala	Pemeriksaan dan tindakan
Compose tidak tersambung	Periksa Docker Engine/Codespaces selesai start, lalu <code>docker info</code> . Di Windows gunakan Git Bash/WSL yang terhubung ke Engine.
<code>bind: address already in use</code>	Hentikan service lokal yang memakai 8080, 3000, atau 5432; ulangi Compose.
API/DB belum healthy	<code>docker compose ps -a</code> dan <code>docker compose logs api db</code> ; setelah perubahan jaringan pakai <code>--force-recreate</code> .
nginx gagal setelah <code>https.conf</code> dibuat	Pastikan sertifikat dibuat dahulu; baca <code>docker compose logs web</code> .
Browser memperingatkan HTTPS	Sertifikat self-signed belum dipercaya browser; verifikasi lab dengan <code>curl --cacert</code> dari toolbox.
Alat jaringan tidak ada di host	Jalankan <code>dig</code> , <code>nmap</code> , dan <code>openssl</code> dari toolbox.
Actions merah	Buka nama job: <code>student</code> perlu identitas, challenge merah pada starter; jika <code>stack</code> merah, baca log build dan smoke.

Pengumpulan dan penutupan

Mahasiswa menyalin `hasil/TEMPLATE_LAPORAN.md` menjadi `hasil/lab03.md`, menyimpan screenshot **milik sendiri** di `hasil/bukti/lab03/`, mengisi `student.json` tanpa NIM, memeriksa `git diff --cached`, lalu commit/push. Minta tautan repo pribadi, commit akhir, dan run Actions. Konfigurasi serta laporan boleh masuk Git; `web/certs/lab.key`, `.env`, token, password nyata, dan screenshot kredensial tidak boleh masuk. Tutup dengan `docker compose down`; `down -v` hanya untuk reset data yang memang diinginkan.